Parte A — Decisiones del equipo (15 min, sin IA, en papel)

El código base ya tomó algunas decisiones de diseño y dejó otras abiertas. Antes de programar, el equipo justifica las tomadas y decide las abiertas. Cada rol redacta sus filas, pero el equipo entero las discute. La justificación debe nombrar la información o la relación concreta, no solo el principio.

#	Rol	Decisión	Estado	Lo que escribe el equipo
D1	Hipotecas	El subsidio semanal lo calcula `Mortgage`	Tomada	Principio GRASP y qué datos de `Mortgage` lo justifican **Respuesta:** El principio GRASP: Experto en información. La clase Mortgage debe calcular el subsidio semanal porque tiene la información necesaria para realizar esta operación: el pago semanal de la hipoteca, los impuestos, el seguro y los ingresos semanales de la pareja. Por lo tanto, es la clase que conoce los datos necesarios para determinar cuánto debe cubrir la fundación.
D2	Hipotecas	¿El subsidio y el aporte de la pareja se guardan como atributos al construir la hipoteca, o se calculan cada vez que se piden?	Abierta	Opción elegida y por qué **Respuesta:** Se decide calcular el subsidio y el aporte de la pareja cada vez que se soliciten, en lugar de almacenarlos como atributos. Esto se debe a que ambos valores se obtienen a partir de otros datos que ya tiene la clase Mortgage. De esta manera, se evita almacenar información repetida y se mantiene la coherencia de los cálculos.
D3	Inversiones	`Investment` está declarada `final`	Tomada, revisable	¿La mantienen así? Una ventaja y un riesgo **Respuesta:** Una ventaja es que da seguridad y garantiza que las reglas de negocio no se alteren. Como es una clase que maneja dinero, al ser final evitamos que alguien cree una subclase tramposa o con errores que cambie la forma en que se calcula el rendimiento semanal. Un riesgo es que se pierde flexibilidad a futuro. Si el día de mañana la fundación empieza a manejar distintos tipos específicos de inversiones y quiere que hereden de Investment para reutilizar código, no se podrá hacer sin quitar el Final.
D4	Registro	¿Quién detecta un número de hipoteca repetido: `RegisterMortgage` o el repositorio?	Abierta	Opción elegida, principio y qué pasaría si mañana cambia el repositorio **Respuesta:** Opción elegida: El repositorio valida la existencia (ej. existsById) y RegisterMortgage gestiona la regla de negocio. Principio: Experto en Información y Bajo Acoplamiento. Si cambia el repositorio: El caso de uso permanece intacto al depender de una interfaz y no del motor de almacenamiento.

#	Rol	Decisión	Estado	Lo que escribe el equipo
D5	Integrador	`EstimateFundsForWeek` recibe `MortgageRepository` (una interfaz) y no `InMemoryMortgageRepository`	Tomada	Principio GRASP y qué gana el caso de uso con esa decisión **Respuesta:** Respuesta: Principio GRASP: Bajo Acoplamiento y Polimorfismo. Ganancia: Desacopla la lógica de negocio de la infraestructura y permite usar mocks para pruebas unitarias.
D6	Integrador	Las sumas sobre todas las hipotecas e inversiones están en métodos privados de `EstimateFundsForWeek`. ¿Se quedan ahí o pasan a un experto nuevo del dominio?	Tomada, revisable	Opción elegida y por qué **Respuesta:** Respuesta: Opción elegida: Pasan a un nuevo experto del dominio (ej. WeeklyFundConsolidator). Por qué: Mantiene Alta Cohesión en el caso de uso (solo orquesta) y permite reutilizar los cálculos en otras partes del sistema.